

Nome: Eduardo Lucas Lemes Januário CT3037568
Nome: Guilherme Batista CT3037274

Análise de Complexidade - Estrutura de Dados 2

Relatório da Prática de Análise de Complexidade de Algoritmos

Link colab:

<https://colab.research.google.com/drive/1-OryUrp7MDpnaeKWI-B8UMWAN028x0Sr?usp=sharing>

Algoritmo 1 - Soma de números de 1 a n

O problema consiste em calcular a soma . Existem três abordagens clássicas: fórmula fechada, iteração e recursão. Elas servem para contrastar custos de $O(1)$ (fórmula) versus $O(n)$ (loop/recursão) e ilustrar a diferença prática entre crescimento constante e linear.

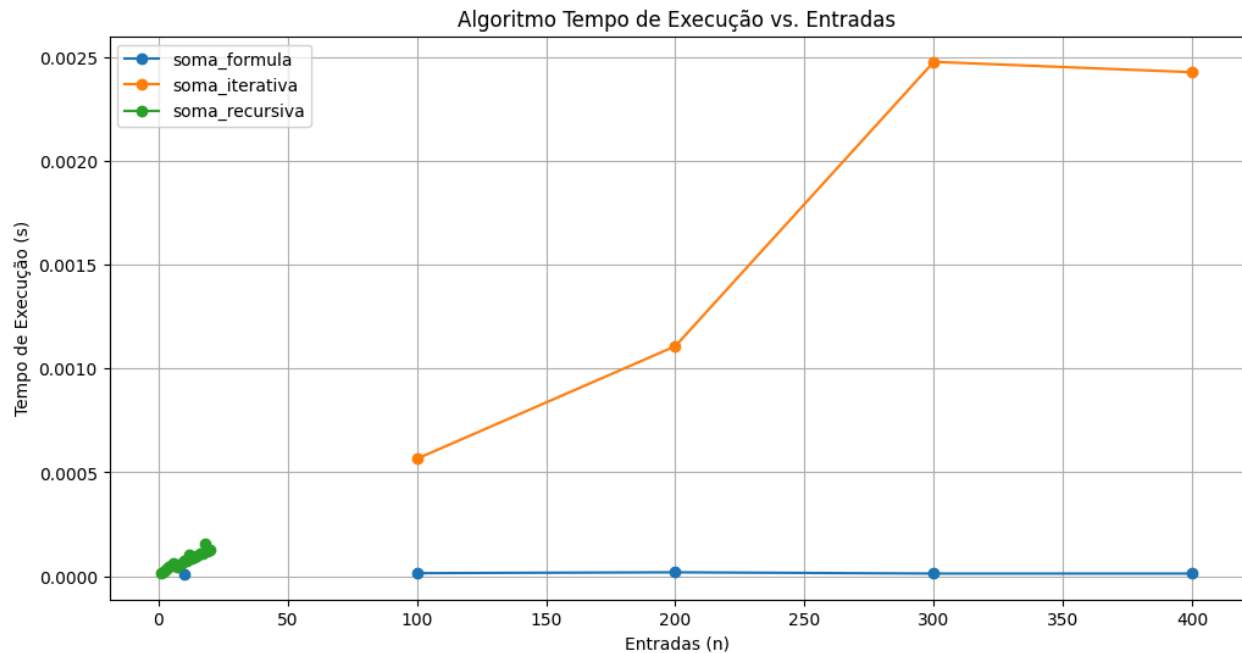
- Fórmula direta: . Envolve poucas operações aritméticas, independentes de . Complexidade de tempo: $O(1)$.
- Iterativo: acumula a soma em um laço de 1 a n : $s = 0$; for i in $1..n$: $s += i$. Complexidade de tempo: $O(n)$.
- Recursivo: $S(n) = n + S(n-1)$ com caso base $S(1)=1$. Complexidade de tempo: $O(n)$ (mas com sobrecarga de chamadas e limites de pilha).

As medições mostram tempos ínfimos para a fórmula, e crescimento linear para o método iterativo/recursivo conforme aumenta.

Soma de números de 1 a n				
Teste	Entradas	soma_formula	soma_iterativa	soma_recursiva
0	1	NaN	NaN	0.000014
1	2	NaN	NaN	0.000023
2	3	NaN	NaN	0.000033
3	4	NaN	NaN	0.000046
4	5	NaN	NaN	0.000049

5	6	NaN	NaN	0.000063
6	7	NaN	NaN	0.000046
7	8	NaN	NaN	0.000056
8	9	NaN	NaN	0.000062
9	10	0.000011	0.000073	0.000072
10	11	NaN	NaN	0.000071
11	12	NaN	NaN	0.000101
12	13	NaN	NaN	0.000083
13	14	NaN	NaN	0.000089
14	15	NaN	NaN	0.000098
15	16	NaN	NaN	0.000107
16	17	NaN	NaN	0.000111
17	18	NaN	NaN	0.000155
18	19	NaN	NaN	0.000121
19	20	NaN	NaN	0.000127
20	100	0.000014	0.000565	NaN
21	200	0.000019	0.001107	NaN
22	300	0.000013	0.002476	NaN
23	400	0.000013	0.002426	NaN

Gráfico comparativo das três abordagens.



- Entre abordagens: a fórmula é invariavelmente mais rápida (tempo constante). O método iterativo cresce linearmente e tende a superar a recursão devido à menor sobrecarga (chamadas de função, frames de pilha). Para valores muito grandes, a recursiva pode ainda enfrentar *stack overflow*.
- Entre máquinas: espera-se variação nos tempos absolutos (CPU, cache, frequência, SO, versão da VM/interpretador), mas o perfil de crescimento se mantém: fórmula iteração recursão. Em máquinas com otimizações de chamada de função ou JIT, a diferença entre iteração e recursão pode diminuir, mas o ranqueamento geral persiste.

Para qualquer , use a fórmula ($O(1)$). Quando a fórmula não se aplica ao problema (por exemplo, somas condicionais), prefira iteração. A recursão é adequada apenas como exercício didático, pois agrega sobrecarga sem benefício prático neste caso.

Algoritmo 2 - Fatorial de um número

O cálculo de contrasta iteração e recursão com a mesma ordem assintótica $O(n)$, mas custos constantes diferentes. É um caso útil para observar limites de pilha e o impacto de *overhead* de chamadas.

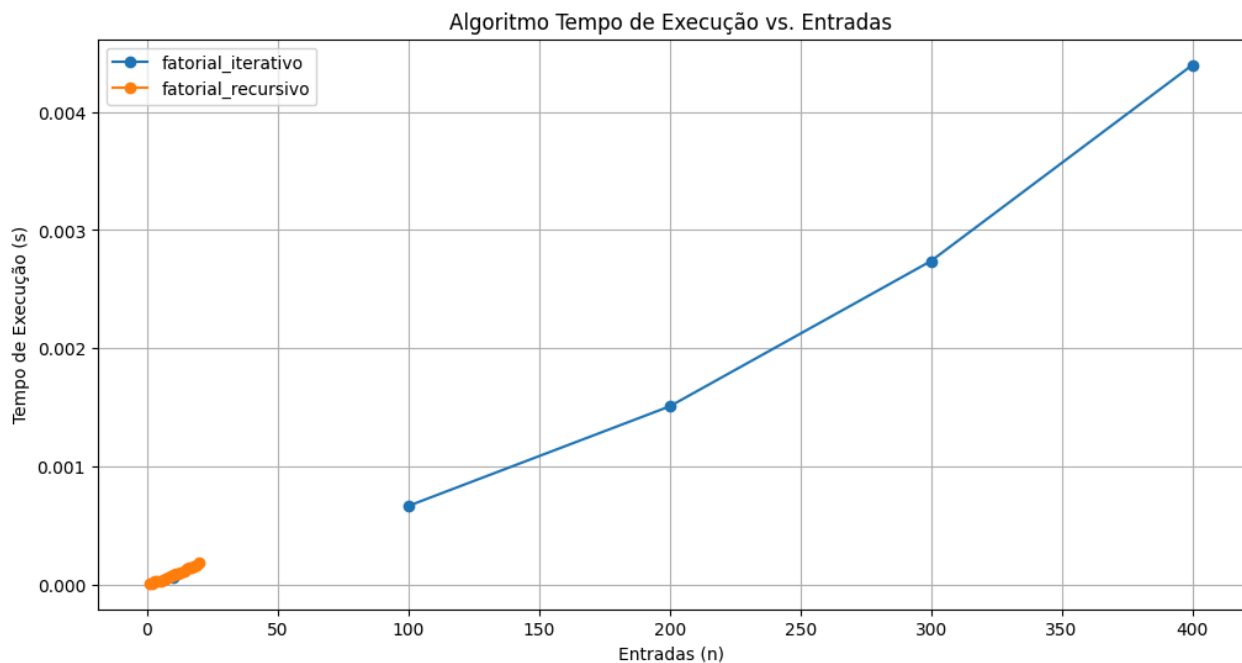
- Iterativo: $f = 1$; for i in $2..n$: $f *= i$. Complexidade $O(n)$.
- Recursivo: $F(n) = n * F(n-1)$ com $F(0)=1$. Complexidade $O(n)$, porém com sobrecarga de pilha. Em linguagens sem otimização de *tail call*, o limite prático de n é o limite de profundidade da pilha.

Os tempos crescem aproximadamente de forma linear com n na versão iterativa. A versão recursiva funciona para valores pequenos/médios e deixa de aparecer nas medições para grandes devido a limites de pilha.

Teste	Entradas	fatorial_iterativo	fatorial_recursoivo
0	1	NaN	0.000014
1	2	NaN	0.000016
2	3	NaN	0.000031
3	4	NaN	0.000030
4	5	NaN	0.000036
5	6	NaN	0.000045
6	7	NaN	0.000056
7	8	NaN	0.000064
8	9	NaN	0.000070
9	10	0.000064	0.000081
10	11	NaN	0.000092
11	12	NaN	0.000098
12	13	NaN	0.000103
13	14	NaN	0.000114
14	15	NaN	0.000140
15	16	NaN	0.000149
16	17	NaN	0.000151
17	18	NaN	0.000157
18	19	NaN	0.000170
19	20	NaN	0.000185
20	100	0.000668	NaN
21	200	0.001510	NaN

22	300	0.002738	NaN
23	400	0.004394	NaN

Comparativo entre iteração e recursão.



- Entre abordagens: iterativo é consistentemente mais rápido e robusto para grandes; recursivo sofre com *overhead* e limites de pilha. Em ambientes com *tail-call optimization*, a diferença pode reduzir, mas ainda há custo por ativação.
- Entre máquinas: CPUs com maior *throughput* de multiplicação e melhores caches tendem a reduzir tempos, mas o crescimento permanece linear. A versão recursiva é a mais sensível ao ambiente (profundidade padrão de pilha, custo de chamadas).

Para produção, use a versão iterativa. A recursiva é útil didaticamente e para problemas em que a recursão modela naturalmente a solução, mas aqui não há vantagem prática.

Algoritmo 3 - Busca linear de um número

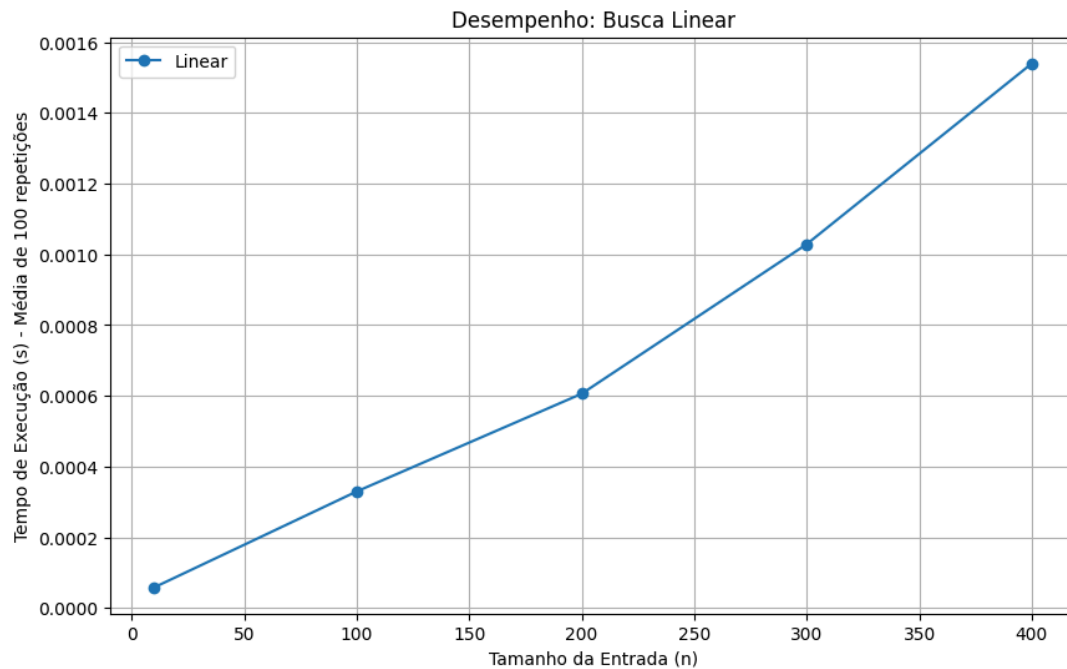
Localizar um elemento em uma coleção desordenada exige inspecionar elementos até encontrar o alvo ou esgotar a lista. A busca linear é a abordagem mais simples e universal.

Percorre sequencialmente o arranjo: for i in $0..n-1$: if $a[i] == x$: return i . No pior caso, consulta todos os elementos. Complexidade de tempo: $O(n)$ (pior/médio caso), $O(1)$ no melhor caso (acerto na primeira posição).

Medições evidenciam crescimento aproximadamente linear com o tamanho do vetor.

Teste	Entrada	busca_linear
0	10	0.00005865
1	100	0.00032983
2	200	0.00060614
3	300	0.00102898
4	400	0.00153950

Curva de tempo.



Versus busca binária: a linear é muito mais lenta em coleções grandes quando os dados podem ser mantidos ordenados. Entretanto, ela não exige ordenação prévia.

- Entre máquinas: diferenças absolutas dependem de memória/cache e largura de banda de acesso; como o padrão é *scan* sequencial, prefetch e caches podem ajudar, mas o crescimento assintótico ($O(n)$) permanece.

Use busca linear quando: (1) a coleção é pequena; (2) não compensa ordenar; (3) acessos são raros e a atualização é frequente. Caso contrário, considere manter a coleção ordenada e usar busca binária.

Algoritmo 4 - Busca binária de um número

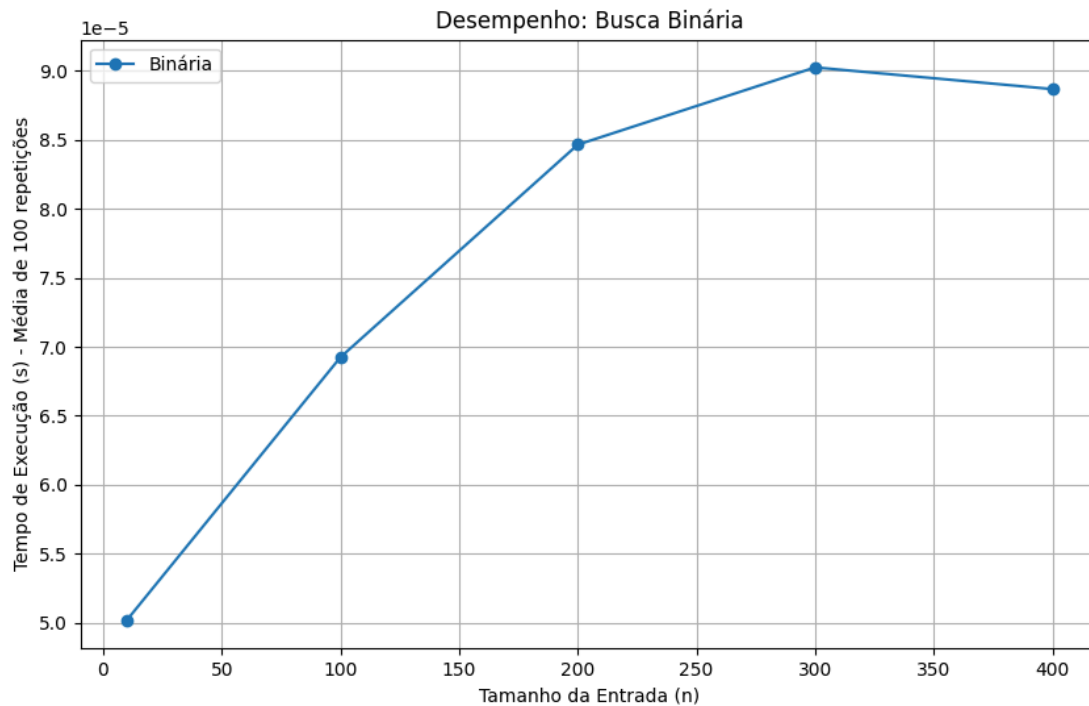
Em coleções ordenadas, a busca binária reduz drasticamente o número de comparações ao dividir o espaço de busca pela metade a cada passo.

Invariantes de intervalo $[l, r]$ e comparação com o elemento médio. A cada iteração, descarta metade do espaço de busca. Complexidade: $O(\log n)$.

Os tempos crescem muito lentamente com n , refletindo o comportamento logarítmico

Teste	Entrada	busca_binaria
0	10	0.00005019
1	100	0.00006924
2	200	0.00008464
3	300	0.00009024
4	400	0.00008867

Curva de tempo em função de



Versus busca linear: para grandes, a binária vence por ampla margem, desde que os dados estejam ordenados (ou o custo de mantê-los ordenados seja aceitável frente ao volume de consultas).

- Entre máquinas: o benefício relativo tende a se manter; latências de memória têm pouco impacto no número de passos, pois o total de acessos cai com .

É a escolha padrão quando a coleção é ordenada ou pode ser mantida assim. Proporciona ganhos significativos já a partir de tamanhos moderados de n .

Algoritmo 5 - Verifica se um número é primo

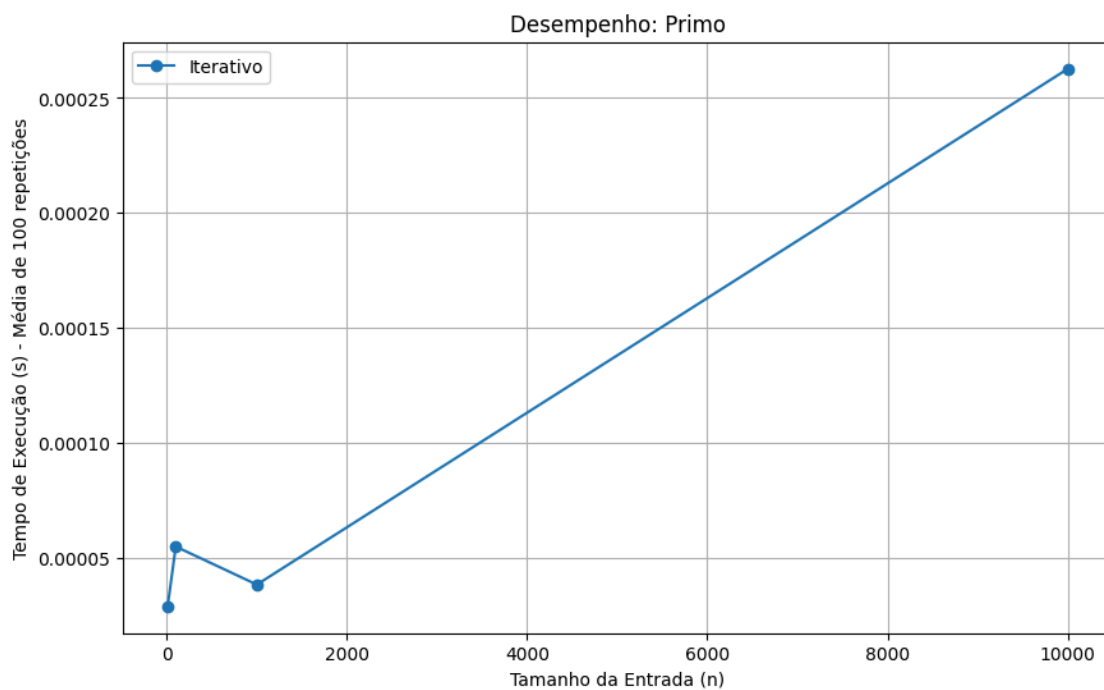
Dado um inteiro m , deseja-se decidir se m é primo. A abordagem direta testa divisores até $\sqrt{m}$.

Para cada inteiro i em $2, 3, \dots, \sqrt{m}$, verifica se i divide m . Se algum dividir, não é primo; caso contrário, é primo. Complexidade por verificação: $O(\sqrt{m})$.

Foram medidos tempos em diferentes faixas de entrada (até 10, 100, 1000 e 10000)

Teste	Entrada	primo_10	primo_100	primo_1000	primo_10000
0	10	0.00002919	NaN	NaN	NaN
1	100	NaN	0.00005513	NaN	NaN
2	200	NaN	NaN	0.00003860	NaN
3	300	NaN	NaN	NaN	0.00026261

Barras com tempo por faixa



Versus crivo/algoritmos avançados: para verificar muitos números em sequência, crivos (Eratóstenes, Atkin) são mais eficientes. Para poucos testes esporádicos, o teste ingênuo até é aceitável.

- Entre máquinas: tempos absolutos variam com a velocidade de divisão/instruções inteiras e memória, porém a tendência de crescimento com permanece.

Adequado para checagens isoladas e valores moderados. Para grandes intervalos ou números muito grandes, prefira crivos (muitos testes) ou testes probabilísticos/determinísticos de primalidade (Miller–Rabin, AKS) conforme o contexto.